

Day 9: Python Part 2 — Variables and Methods

Today, I continued with Python for cybersecurity. This part was about variables and some basic string methods. I focused on using variables as shortcuts, then applying methods like `upper()`, `lower()`, and `title()`, and then working with integers and floats.

Using variables as shortcuts

I started by writing a full sentence as a string. Then I learned that if I want to apply commands to it, I don't want to copy/paste the full sentence every time. That's why variables are useful — they let me store the string once and reuse it.

```
code = "I am the man that my mother named me a lion"
```

To print it fast, I just print the variable name:

```
print(code)
```

String methods (upper, lower, title)

Then I practiced string methods. These methods return a modified version of the string.

```
print(code.upper())
print(code.lower())
print(code.title())
```

Counting characters with len()

To get the number of characters in the string, I used `len()`:

```
print(len(code))
```

Variables types: string, int, float

Next, I wrote shortcuts (variables) for a string, an integer, and a float.

```
name = "Muhammad" # string
age = 16 # int
rate = 98.5 # float
```

I learned that float means it has decimals, and int means no decimals.

Printing values with casting (int, float, str)

To print age as an integer, I used `int(age)`. And to print rate as a float, I used `float(rate)`.

```
print(int(age))  
print(float(rate))
```

Concatenating strings with numbers (using str)

When I tried to combine my name and age in one sentence, I learned I can't concatenate a string with an integer directly. So I converted the number to a string using str(age).

```
print("My name is " + name + " and I am " + str(age) + " years old")
```

Updating numbers (+=)

To add a number to an integer variable, I used +=. For example, adding 2 to age:

```
age += 2  
print(age)
```

And to add another integer variable into age, I created birth = 1, then added it:

```
birth = 1  
age += birth  
print(age)
```

This was Day 9. I'm still learning, but variables made Python feel much cleaner because I can reuse the same values and apply methods on them easily.